Báo cáo BTVN 31/3/2026

Email Request (Bài 1)

Mô tả cho Bài tập

Lập trình ứng dụng server trả về địa chỉ email của sinh viên theo yêu cầu:

- Khi có client kết nối, server gửi thông điệp để hỏi "Họ tên" và "MSSV"
- Từ dữ liệu nhận được, server phản hồi địa chỉ email cho client, theo cấu trúc email của sv ĐHBK
- Sử dụng non-blocking server, thử nghiệm với nhiều client kết nối cùng một lúc

Mã Nguồn

```c
/*****************************************************************
****************
* @file    non_blocking_server_hust.c
* @brief   Server Non-blocking trả về email sinh viên ĐHBK
*****************************************************************
***************/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <netdb.h>
#include <sys/ioctl.h>
#include <errno.h>
#include <ctype.h>

#define MAX_CLIENTS 64

// Lưu trạng thái Client
typedef struct {
    int fd;
    int state; //0: Chưa nhập tên-mssv; 1: chưa nhập mssv
    char name[256];
```

```
} ClientState;

ClientState clients[MAX_CLIENTS];
int nclients = 0;

// Hàm xóa client khỏi mảng khi ngắt kết nối
void remove_client(int index) {
    close(clients[index].fd);
    for (int i = index; i < nclients - 1; i++) {
        clients[i] = clients[i + 1];
    }
    nclients--;
}

void Tao_email(const char* fullname, const char* mssv, char*
email_out) {
    char temp_name[256];
    strncpy(temp_name, fullname, sizeof(temp_name) - 1);
    temp_name[sizeof(temp_name) - 1] = '\0';

    for(int i = 0; temp_name[i]; i++) {
        temp_name[i] = tolower(temp_name[i]);
    }

    char *words[20];
    int count = 0;

    char *token = strtok(temp_name, " ");
    while(token != NULL && count < 20) {
        words[count++] = token;
        token = strtok(NULL, " ");
    }

    if(count == 0) {
        strcpy(email_out, "invalid_name@sis.hust.edu.vn");
        return;
    }

    char initials[20] = "";
    for(int i = 0; i < count - 1; i++) {
        strncat(initials, words[i], 1);
```

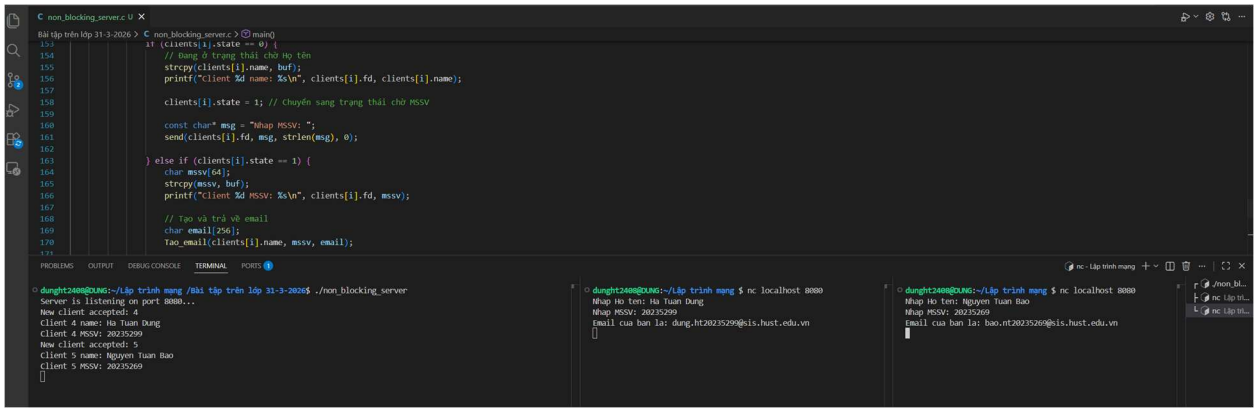
```
    }

    sprintf(email_out, "%s.%s%s@sis.hust.edu.vn", words[count -
1], initials, mssv);
}

int main() {
    int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (listener == -1) {
        perror("socket() failed");
        return 1;
    }

    // Chuyển socket listener sang non-blocking
    unsigned long ul = 1;
    ioctl(listener, FIONBIO, &ul);

    if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR,
&(int){1}, sizeof(int))) {
        perror("setsockopt() failed");
        close(listener);
        return 1;
    }

    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(8080);

    if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)))
{
        perror("bind() failed");
        close(listener);
        return 1;
    }

    if (listen(listener, 5)) {
        perror("listen() failed");
        close(listener);
        return 1;
    }
```

```
    printf("Server is listening on port 8080...\n");

    char buf[256];
    int len;

    while (1) {
        // Chấp nhận kết nối
        int client = accept(listener, NULL, NULL);
        if (client == -1) {
            if (errno != EWOULDBLOCK) {
                perror("accept() error");
            }
        } else {
            printf("New client accepted: %d\n", client);

            ul = 1;
            ioctl(client, FIONBIO, &ul);

            // Lưu vào mảng quản lý
            clients[nclients].fd = client;
            clients[nclients].state = 0;
            memset(clients[nclients].name, 0,
sizeof(clients[nclients].name));
            nclients++;

            char* msg = "Nhap Ho ten: ";
            send(client, msg, strlen(msg), 0);
        }

        // Nhận dữ liệu từ client
        for (int i = 0; i < nclients; i++) {
            len = recv(clients[i].fd, buf, sizeof(buf) - 1, 0);
            if (len == -1) {
                if (errno != EWOULDBLOCK) {
                    printf("Client %d error. Disconnecting.\n",
clients[i].fd);
                    remove_client(i);
                    i--; // Lùi index vì mảng đã bị dồn lên
                }
            } else if (len == 0) {
```

```
                printf("Client %d disconnected.\n",
clients[i].fd);
                remove_client(i);
                i--;
            } else {
                buf[len] = '\0';

                // Loại bỏ ký tự \r và \n ở cuối chuỗi
                buf[strcspn(buf, "\r\n")] = 0;

                if(strlen(buf) == 0) continue;

                if (clients[i].state == 0) {
                    // Đang ở trạng thái chờ Họ tên
                    strcpy(clients[i].name, buf);
                    printf("Client %d name: %s\n",
clients[i].fd, clients[i].name);

                    clients[i].state = 1; // Chuyển sang trạng
thái chờ MSSV

                    const char* msg = "Nhap MSSV: ";
                    send(clients[i].fd, msg, strlen(msg), 0);

                } else if (clients[i].state == 1) {
                    char mssv[64];
                    strcpy(mssv, buf);
                    printf("Client %d MSSV: %s\n",
clients[i].fd, mssv);

                    // Tạo và trả về email
                    char email[256];
                    Tao_email(clients[i].name, mssv, email);

                    char response[512];
                    sprintf(response, "Email cua ban la: %s\n",
email);
                    send(clients[i].fd, response,
strlen(response), 0);

                    // //Ngắt kết nối client
```

```
                    // printf("Completed request for client %d.
Disconnecting.\n", clients[i].fd);
                    // remove_client(i);
                    // i--;
                }
            }
        }

        usleep(1000);
    }

    close(listener);
    return 0;
}
```

Hình ảnh 1.1:



- **Phía Server (bên trái):** Server đã bắt được kết nối và xử lý đồng thời hai client riêng biệt (được đánh dấu fd lần lượt là 4 và 5). Quá trình non-blocking hoạt động chính xác khi hệ thống ghi nhận đầy đủ luồng dữ liệu đầu vào gồm "Họ tên" và "MSSV" từ từng client.
- **Phía Client (bên phải):** Hai tiến trình client độc lập được kết nối thông qua lệnh trình giả lập netcat. Cả hai tiến trình đều hoàn thành tốt chu trình tương tác: nhận thông điệp yêu cầu từ server, gửi dữ liệu đầu vào, và nhận phản hồi trả về là địa chỉ email được định dạng chuẩn xác theo cấu trúc của ĐHBK.